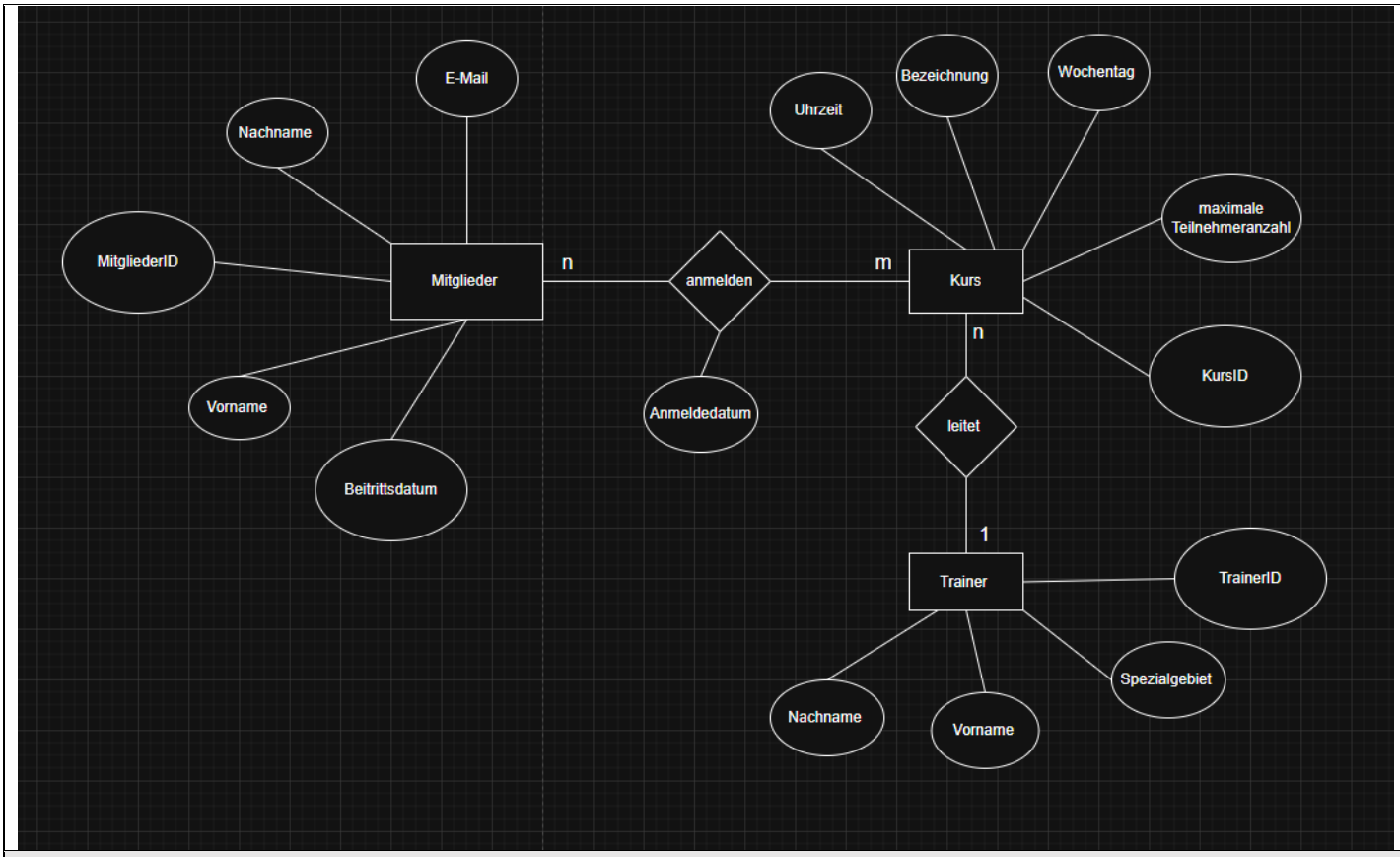
Name: Kenan Pipic **Datum:** 08.04.2026 **Klasse:** 3AHIF

3. Grafische Leistungsfeststellung – DBI1
Thema: Fitnessstudio-Verwaltung (ERM, RM, Python/SQLite3, Anvil)

Punkte	**/ 100**				
Note	100-87.5 □ Sehr gut	< 87.5-75 □ Gut	< 75-62.5 □ Befriedigend	< 62.5-50 □ Genügend	< 50 □ Nicht Genügend
Unterschrift					
GitHub-Link					
Anvil Publish-Link					

Form	**Pun**
1. Form der Arbeit (Leserlich, vollständige Abgabe, sinnvolle Dateinamen, Ausführbar, Fehlerbehandlung)	
Szenario: Fitnessstudio-Verwaltung	
Ein lokales Fitnessstudio möchte seine Mitgliederverwaltung und Kursbuchungen digitalisieren. Es gelten folgende Rahmenbedingungen: • Das Fitnessstudio bietet verschiedene Kurse an (z. B. Yoga, Pilates, Spinning, Zumba). Von jedem Kurs werden Bezeichnung, Wochentag, Uhrzeit und maximale Teilnehmeranzahl gespeichert. • Jeder Kurs wird von genau einem Trainer geleitet. Ein Trainer kann mehrere Kurse leiten. Von jedem Trainer werden Vorname, Nachname und Spezialgebiet erfasst. • Mitglieder können sich für beliebig viele Kurse anmelden. Für jeden Kurs können sich mehrere Mitglieder einschreiben. Bei jeder Anmeldung wird das Anmeldedatum gespeichert. Von jedem Mitglied werden Vorname, Nachname, E-Mail-Adresse und Beitrittsdatum erfasst.	
1.1. Aufgabe 1 – Entity-Relationship-Modell (ERM)	**Pun**
Erstelle auf Basis des Szenarios ein vollständiges ERM: Füge das fertige ERM als Bild in dieses Dokument ein. **(draw.io-Export)**	——
[ERM-Bild hier einfügen]	

Name: Kenan Pipic **Datum:** 08.04.2026 **Klasse:** 3AHIF

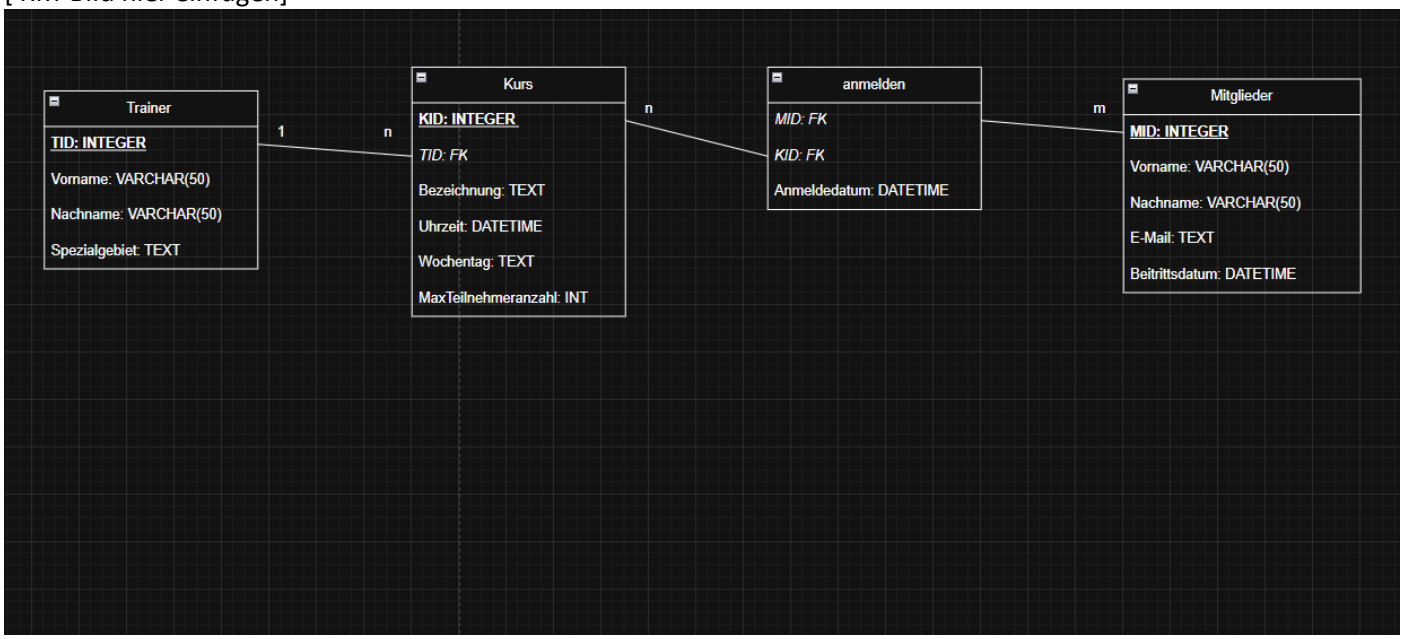


1.2. Aufgabe 2 – Relationales Modell (RM)

Pun

Leite aus deinem ERM ein vollständiges Relationales Modell ab: Füge das fertige RM als Bild hier ein. **(draw.io-Export)**

[RM-Bild hier einfügen]



1.3. Aufgabe 3 – Python-Skript mit SQLite3 ★ KI-Tools erlaubt

Pun

Schreibe ein Python-Skript Nachname_Vorname_fitnessstudio.py, das:

1. Eine SQLite-Datenbank Nachname_Vorname_fitnessstudio.db erstellt
2. Alle Tabellen aus dem RM anlegt
3. Sinnvolle Beispieldaten einfügt:
   ≥ 3 Trainer • ≥ 5 Kurse • ≥ 6 Mitglieder • ≥ 8 Anmeldungen

1.4. Aufgabe 4 – Anvil-Implementierung

Pun

Implementiere die Fitnessstudio-Verwaltung als Anvil-Webanwendung (anvil.works).

Name: Kenan Pipic **Datum:** 08.04.2026 **Klasse:** 3AHIF

Pflichtfunktionen:

1. Kursübersicht – Alle Kurse inkl. Trainername

Kurs	Wochentag	Uhrzeit	Trainer	Teilnehmer	Anmelden
Morning Yoga	Montag	08:00	Berger Anna	2/15	Anmelden
Power Lifting	Dienstag	17:00	Huber Markus	2/10	Anmelden
Pilates Basics	Mittwoch	10:00	Mayer Sophie	2/12	Anmelden
HIIT Cardio	Donnerstag	18:30	Schneider Leon	1/20	Anmelden
Abend Yoga	Freitag	19:00	Berger Anna	2/15	Anmelden
Rückenfit	Samstag	09:00	Huber Markus	1/10	Anmelden

2. Neue Anmeldung

Mitglied	Anmelden
Bauer Thomas	Anmelden
Fuchs Stefan	Anmelden
Schwarz Maria	Anmelden
Weiß David	Anmelden
Braun Nina	Anmelden

Bonusfunktionen (optional):

- Mitglieder in einem Kurs anzeigen

1.5. Aufgabe 5 - Abgabe

1. Nachname Vorname in der Kopfzeile eintragen
2. GitHub-Link oben eintragen
3. Anvil Publish-Link oben eintragen
4. Folgende Dateien ins Git-Repo pushen
    a. Nachname_Vorname_ERM.draw.io
    b. Nachname_Vorname_RM.draw.io
    c. Nachname_Vorname_fitnessstudio.py
5. Code hier einfügen

Nachname_Vorname_fitnessstudio.py

```
import sqlite3


# Verbindung zur DB (Datei wird erstellt)

conn = sqlite3.connect("Pipic_Kenan_fitnessstudio.db")

cursor = conn.cursor()


# Tabellen erstellen

cursor.execute("""

CREATE TABLE Trainer (

  TID INTEGER PRIMARY KEY,

  Vorname VARCHAR(50),

  Nachname VARCHAR(50),

  Spezialgebiet TEXT
```

```
)
""")

cursor.execute("""
CREATE TABLE Mitglieder (
   MID INTEGER PRIMARY KEY,
   Vorname VARCHAR(50),
   Nachname VARCHAR(50),
   Email TEXT,
   Beitrittsdatum DATETIME
)
""")

cursor.execute("""
CREATE TABLE Kurs (
   KID INTEGER PRIMARY KEY,
   TID INTEGER,
   Bezeichnung TEXT,
   Uhrzeit DATETIME,
   Wochentag TEXT,
   MaxTeilnehmeranzahl INT,
   FOREIGN KEY (TID) REFERENCES Trainer(TID)
)
""")

cursor.execute("""
CREATE TABLE Anmelden (
   MID INTEGER,
   KID INTEGER,
   Anmeldedatum DATETIME,
   PRIMARY KEY (MID, KID),
   FOREIGN KEY (MID) REFERENCES Mitglieder(MID),
   FOREIGN KEY (KID) REFERENCES Kurs(KID)
)
""")
```

Name: Kenan Pipic **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
# ========================
# Daten einfügen
# ========================

# Trainer
trainer = [
    (1, "Max", "Müller", "Fitness"),
    (2, "Anna", "Schmidt", "Yoga"),
    (3, "Lukas", "Meier", "Cardio")
]
cursor.executemany("INSERT INTO Trainer VALUES (?, ?, ?, ?)", trainer)

# Mitglieder
mitglieder = [
    (1, "Amar", "Pipic", "amar@mail.com", "2026-01-01"),
    (2, "Lisa", "Huber", "lisa@mail.com", "2026-02-10"),
    (3, "Tom", "Keller", "tom@mail.com", "2026-03-15")
]
cursor.executemany("INSERT INTO Mitglieder VALUES (?, ?, ?, ?, ?)", mitglieder)

# Kurse
kurse = [
    (1, 1, "Krafttraining", "2026-04-10 18:00", "Montag", 10),
    (2, 2, "Yoga Anfänger", "2026-04-11 17:00", "Dienstag", 15),
    (3, 3, "HIIT", "2026-04-12 19:00", "Mittwoch", 12)
]
cursor.executemany("INSERT INTO Kurs VALUES (?, ?, ?, ?, ?, ?)", kurse)

# Anmeldungen
anmeldungen = [
    (1, 1, "2026-04-01"),
    (1, 2, "2026-04-02"),
    (2, 2, "2026-04-03"),
    (3, 3, "2026-04-04")
```

```
]
cursor.executemany("INSERT INTO Anmelden VALUES (?, ?, ?)", anmeldungen)

# Speichern
conn.commit()

# ========================
# Testabfrage
# ========================
for row in cursor.execute("""
SELECT m.Vorname, m.Nachname, k.Bezeichnung
FROM Anmelden a
JOIN Mitglieder m ON a.MID = m.MID
JOIN Kurs k ON a.KID = k.KID
"""):
    print(row)

conn.close()
```

Anvil Frontend Code

Kursuebersicht:

```
from ._anvil_designer import kursuebersichtTemplate
from anvil import *
import anvil.server
import anvil.tables as tables
import anvil.tables.query as q
from anvil.tables import app_tables


class kursuebersicht(kursuebersichtTemplate):
  def __init__(self, **properties):
    self.init_components(**properties)

    self.repeating_panel_1.items = anvil.server.call('get_kurse')
```

Name: Kenan Pipic **Datum:** 08.04.2026 **Klasse:** 3AHIF

RowTemplate1:

```
from ._anvil_designer import RowTemplate1Template
from anvil import *
from ..anmeldung import anmeldung


class RowTemplate1(RowTemplate1Template):
  def __init__(self, **properties):
    self.init_components(**properties)

    self.label_kurs.text = self.item['bezeichnung']
    self.label_wochentag.text = self.item['wochentag']
    self.label_trainer.text = self.item['trainer']
    self.label_teilnehmer.text = str(self.item['teilnehmer'])

  @handle("button_anmelden", "click")
  def button_anmelden_click(self, **event_args):
    form = get_open_form()
    form.content_panel.clear()
    form.content_panel.add_component(anmeldung(self.item))
```

Anmeldung

```
from ._anvil_designer import anmeldungTemplate
from anvil import *
import anvil.server
import anvil.tables as tables
import anvil.tables.query as q
from anvil.tables import app_tables


class anmeldung(anmeldungTemplate):
  def __init__(self, **properties):
    # Set Form properties and Data Bindings.
    self.init_components(**properties)
```

```
  self.repeating_panel_1.items = anvil.server.call('get_Mitglieder')
```

RowTemplate2:

```
from ._anvil_designer import RowTemplate2Template
from anvil import *
import anvil.server
import anvil.tables as tables
import anvil.tables.query as q
from anvil.tables import app_tables


class RowTemplate2(RowTemplate2Template):
 def __init__(self, **properties):
  # Set Form properties and Data Bindings.
  self.init_components(**properties)

  # Any code you write here will run before the form opens.
  self.label_teilnehmer.text = self.item['Mitgleider']
```

Anvil Backend Code

```
import anvil.server
import sqlite3
from anvil.files import data_files


# This is a server module. It runs on the Anvil server,
```

```
# rather than in the user's browser.
#
# To allow anvil.server.call() to call functions here, we mark
# them with @anvil.server.callable.
# Here is an example - you can replace it with your own:
#
# @anvil.server.callable
# def say_hello(name):
#   print("Hello, " + name + "!")
#   return 42
#



def get_connection_dict():
  conn = sqlite3.connect(data_files["Pipic_Kenan_fitnessstudio.db"])
  conn.row_factory = sqlite3.Row
  return conn

@anvil.server.callable
def get_kurse():
  with get_connection_dict() as conn:
    cur = conn.cursor()
    result = cur.execute("""
      SELECT
        k.KID AS kid,
        k.Bezeichnung AS bezeichnung,
        k.Wochentag AS wochentag,
        k.Uhrzeit AS uhrzeit,
        t.Vorname || ' ' || t.Nachname AS trainer,
        k.MaxTeilnehmeranzahl AS max_teilnehmer,
        COUNT(a.MID) AS teilnehmer
      FROM Kurs k
      JOIN Trainer t ON k.TID = t.TID
      LEFT JOIN anmelden a ON k.KID = a.KID
      GROUP BY
```

```
    k.KID,
    k.Bezeichnung,
    k.Wochentag,
    k.Uhrzeit,
    t.Vorname,
    t.Nachname,
    k.MaxTeilnehmeranzahl
  ORDER BY k.Wochentag, k.Uhrzeit
 """).fetchall()

return [dict(row) for row in result]
```